

**MBA
USP
ESALQ**

**ÁRVORES, REDES E ENSEMBLE
MODELS III**

João F. Serrajordia R. de Mello

***A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.**

Proibida a reprodução, total ou parcial, sem autorização. Lei nº 9610/98

Você vai precisar de...



Preparativos

- Abrir o R
- Importar as bibliotecas
- Algo para fazer suas anotações

Agenda



Exercício I (gorjetas)

Exercício II (Imóveis)

Boosting

Grid Search CV

Introdução

Wrap-up

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                  minsplit = 5,  
26                                                  maxdepth = 30))
```

Essa é a variável
resposta, é ela que
queremos “prever”

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                    minsplit = 5,  
26                                                    maxdepth = 30))
```

Essas são as
variáveis
explicativas. É com
elas que vamos
Fazer as
“previsões”

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                    minsplit = 5,  
26                                                    maxdepth = 30))
```

O Data Frame de
treino é indicado
aqui

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                    minsplit = 5,  
26                                                    maxdepth = 30))
```

Esse método indica
que a nossa variável
é qualitativa (no
caso, as opções são
sobreviveu = SIM ou
NÃO)

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                           Pclass +  
16                           Sex + Age +  
17                           SibSp +  
18                           Parch +  
19                           Fare +  
20                           Embarked,  
21                           data=treino,  
22                           method='class',  
23                           xval=5,  
24                           control = rpart.control(cp = 0,  
25                                                       minsplit = 5,  
26                                                       maxdepth = 30))
```

O RPART tem implementado o K-fold como método de validação cruzada.

Esse parâmetro é o “K” do k-fold.

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                    minsplit = 5,  
26                                                    maxdepth = 30))
```

Este parâmetro espera um objeto especial, definido pela função `rpart.control`. Esse objeto especial define algumas opções do algoritmo.

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                    minsplit = 5,  
26                                                    maxdepth = 30)
```

Esta função é do pacote *rpart*. Há outros hiperparâmetros parametrizados. Para mais informações, consulte o help rodando `?rpart.control` no console.

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                           Pclass +  
16                           Sex + Age +  
17                           SibSp +  
18                           Parch +  
19                           Fare +  
20                           Embarked,  
21                           data=treino,  
22                           method='class',  
23                           xval=5,  
24                           control = rpart.control(cp = 0,  
25                                                       minsplit = 5,  
26                                                       maxdepth = 30)
```

Este é o custo de complexidade. Quanto maior for esse valor, mais simples será a árvore.

K-fold no *rpart*

```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                  minsplit = 5,  
26                                                  maxdepth = 30))
```

Este hiperparâmetro indica quantas observações devem haver como mínimo para se procurar uma quebra. Ele é um critério de parada do algoritmo.

K-fold no *rpart*

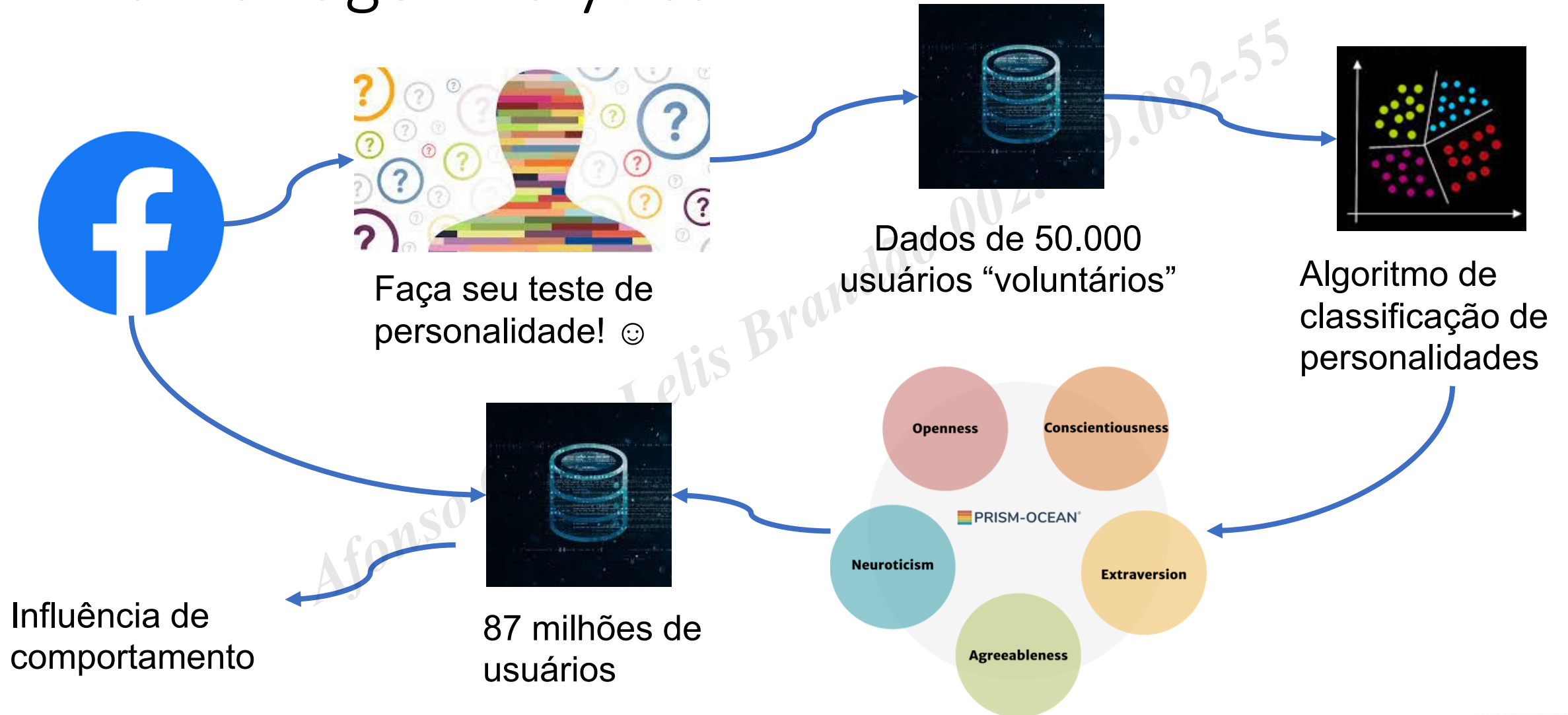
```
14 arvore <- rpart::rpart(Survived ~  
15                          Pclass +  
16                          Sex + Age +  
17                          SibSp +  
18                          Parch +  
19                          Fare +  
20                          Embarked,  
21                          data=treino,  
22                          method='class',  
23                          xval=5,  
24                          control = rpart.control(cp = 0,  
25                                                    minsplit = 5,  
26                                                    maxdepth = 30)
```

Este hiperparâmetro indica a profundidade máxima que a árvore pode ter. Outro critério de parada.

Introdução

Caso Cambridge Analytica

Cambridge Analytica



Exercício

TIPS

Exercício

E tá lembrado da
atividade??

Bora lá??

Camón, dongle?



AREM3_script01_Ex1_aula_anterior.R

Exercício I – comportamento de gorjeta

Queremos entender o comportamento de pagamento de gorjetas em mesas de restaurante.

Será que há uma estratégia ótima na escolha do atendimento de mesas/turnos?



A base

Total_bill: Total da conta

Tip: Gorjeta

Sex: Sexo da pessoa que atende a mesa

Smoker: Indica se há fumantes na mesa

Time: Dia da semana

Day: Almoço ou jantar

Size: Quantidade de pessoas na mesa

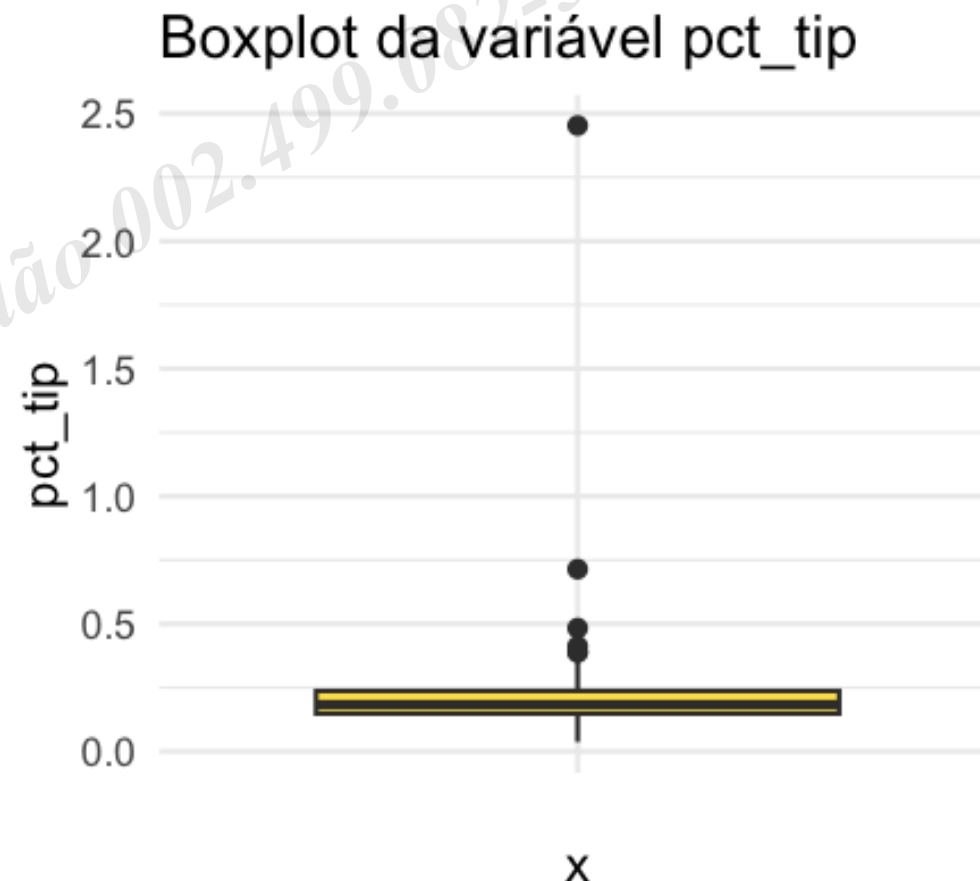
	total_bill	tip	sex	smoker	day	time	size	pct_tip
1	16.99	1.01	Female	No	Sun	Dinner	2	0.06320401
2	10.34	1.66	Male	No	Sun	Dinner	3	0.19124424
3	21.01	3.50	Male	No	Sun	Dinner	3	0.19988578
4	23.68	3.31	Male	No	Sun	Dinner	2	0.16249386
5	24.59	3.61	Female	No	Sun	Dinner	4	0.17206864
6	25.29	4.71	Male	No	Sun	Dinner	4	0.22886297
7	8.77	2.00	Male	No	Sun	Dinner	2	0.29542097

Primeira inspeção na variável resposta

O percentual da gorjeta parece se concentrar entre 10 e 25%.

Há um valor discrepante na casa dos 250%.

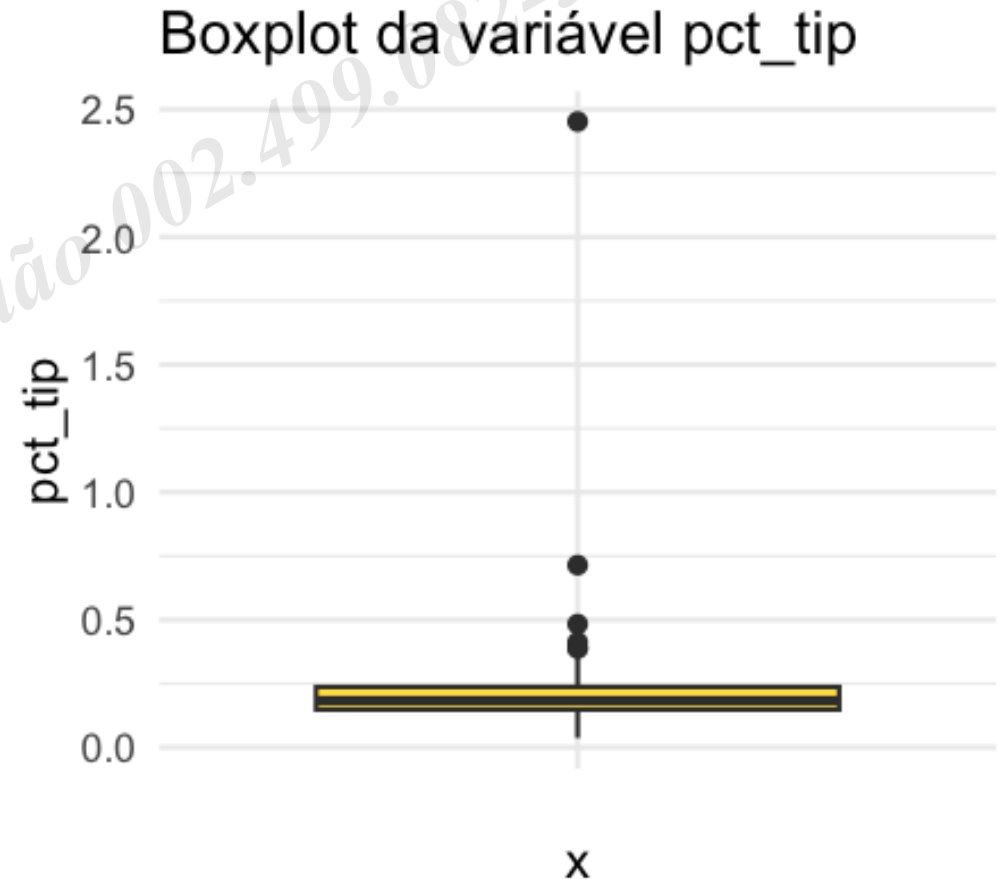
```
18 boxplot(tips$pct_tip)
19 # Criando o boxplot com ggplot2
20 ggplot(tips, aes(x = "", y = pct_tip)) +
21   geom_boxplot(fill = viridis_pal(begin=1)(0.15)) +
22   labs(title = "Boxplot da variável pct_tip") +
23   theme_minimal()
```



O problema com o outlier

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Como a nossa métrica de impureza tem um termo ao quadrado, valores discrepantes podem ter impacto muito grande.



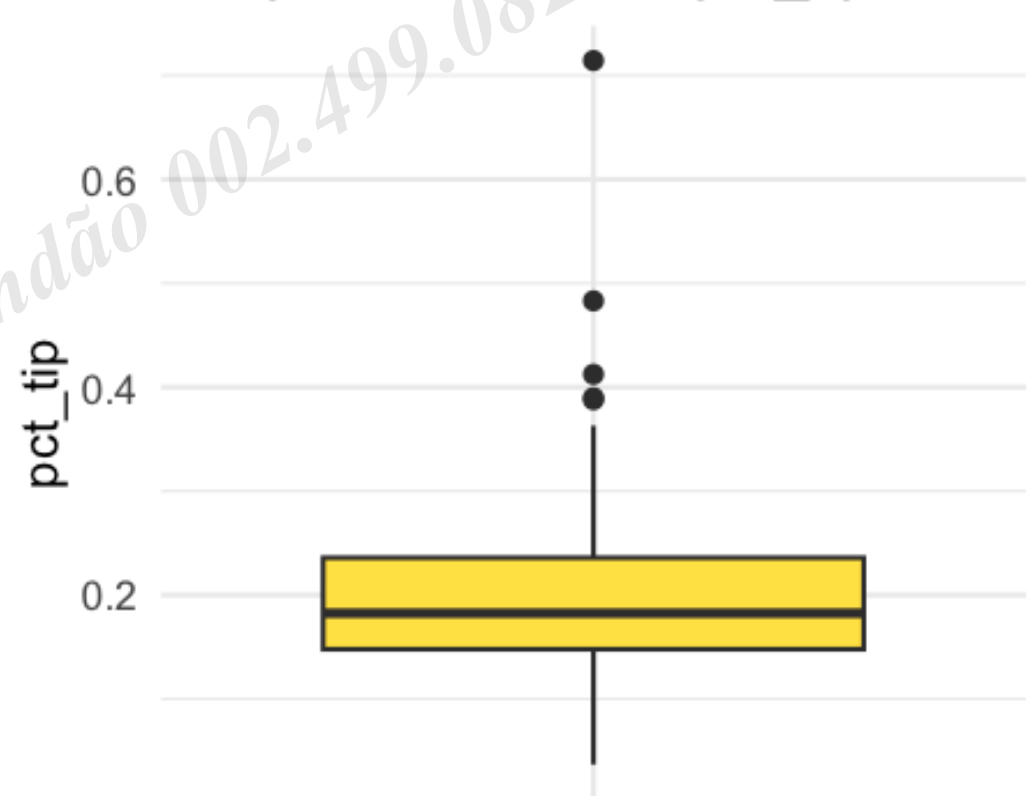
O problema com o outlier

Eliminando o outlier:

```
tips <- tips[tips$pct_tip<1,]
```

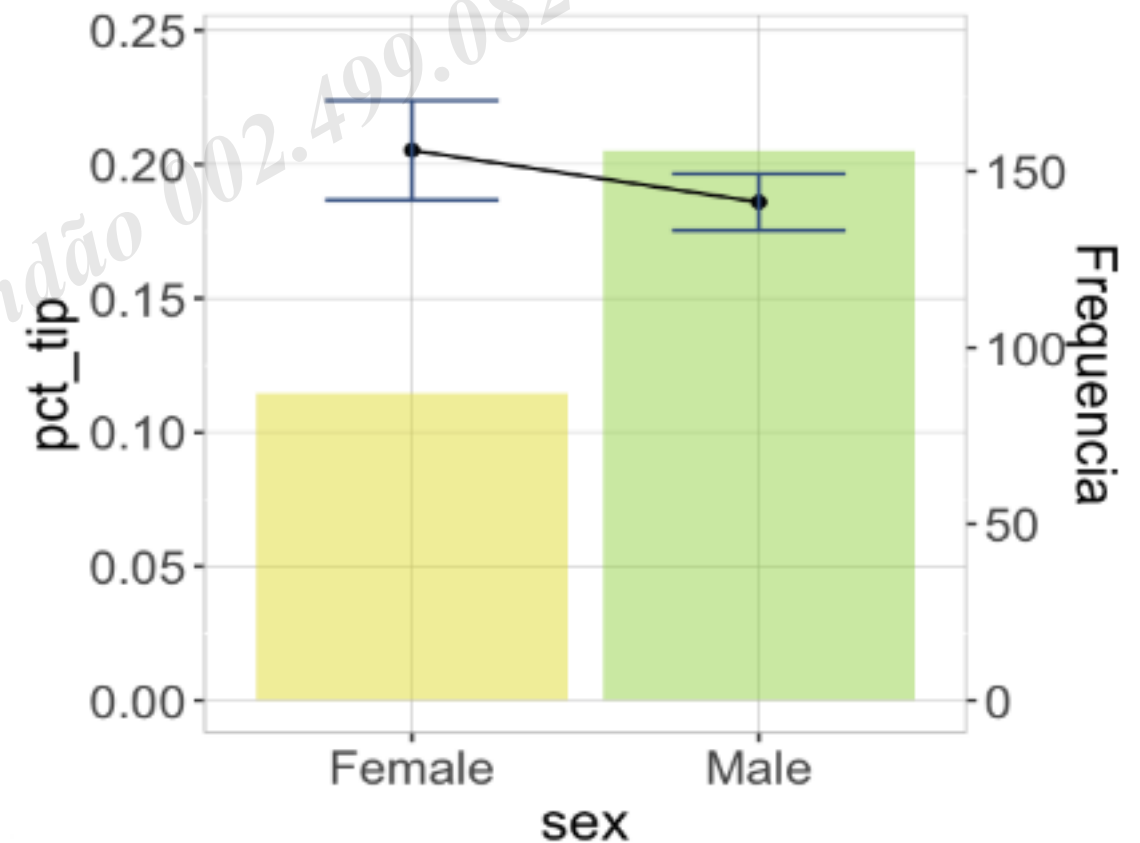
Ainda restam pontos afastados, mas não são tão aberrantes, vamos mantê-los na base.

Boxplot da variável pct_tip



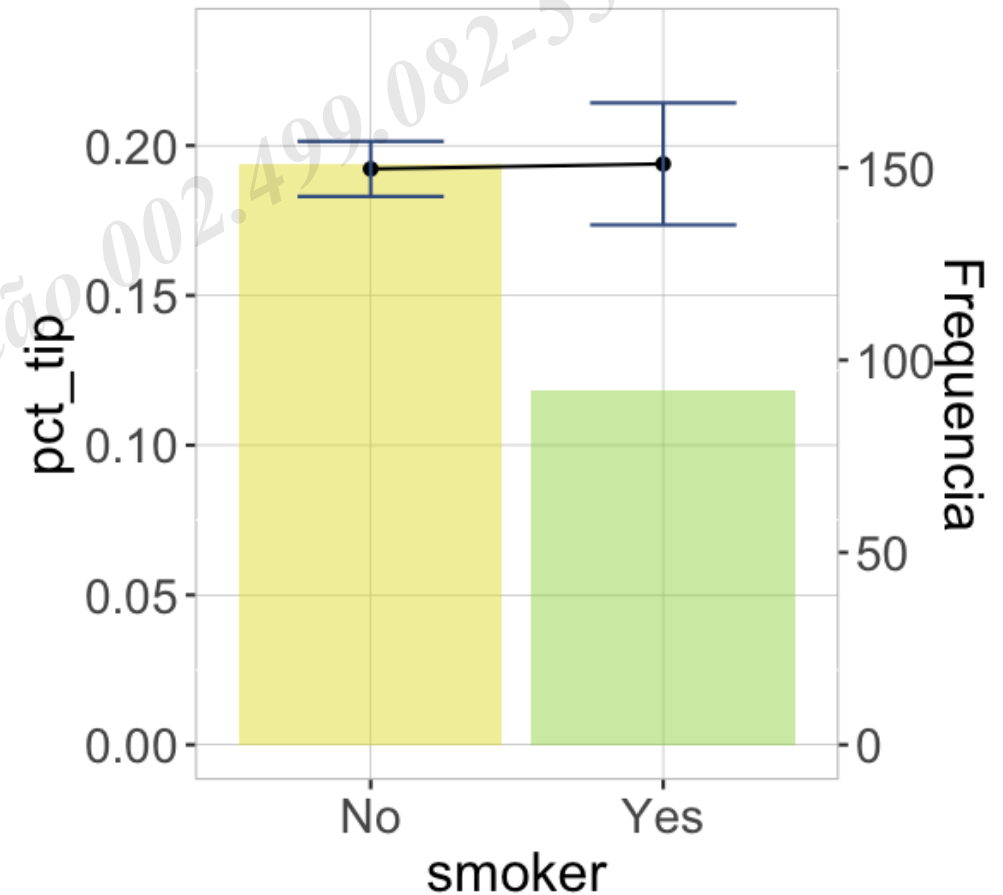
Etapa descritiva

Aparentemente garçonetes ganham mais gorjeta que garçons, mas a diferença é pequena se considerarmos a variabilidade.



Etapa descritiva

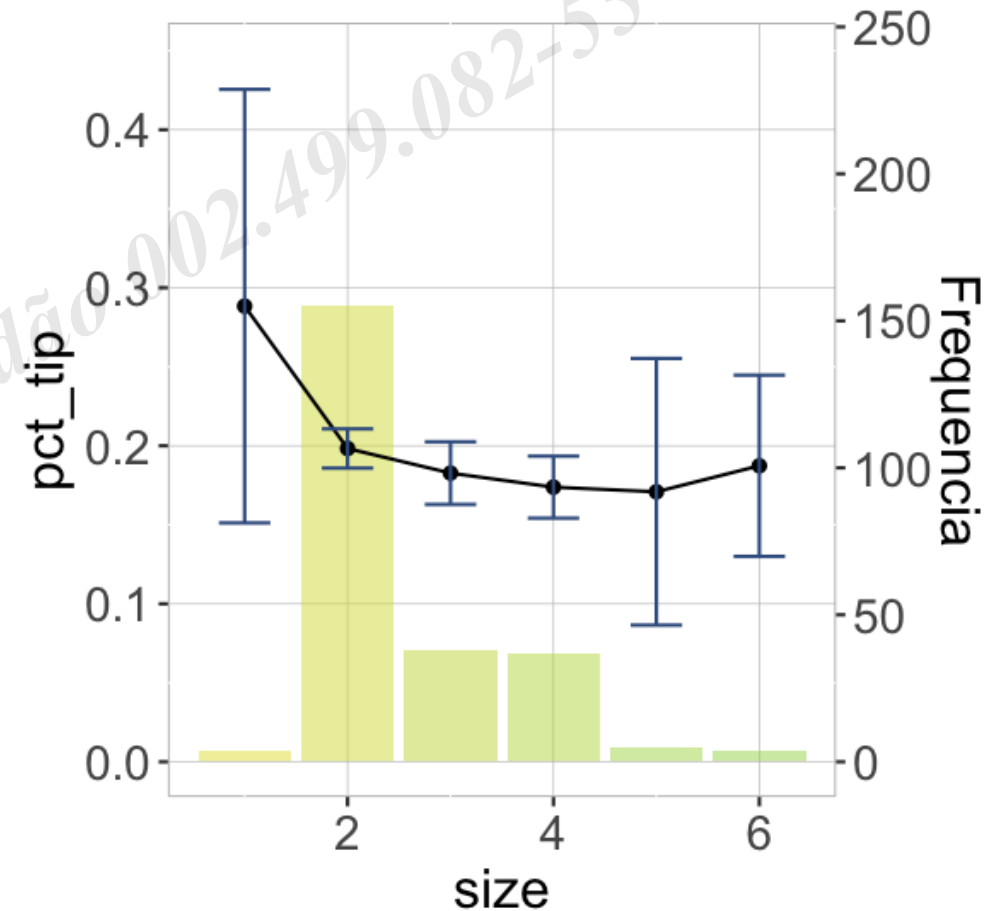
No geral, mesas de fumantes e não fumantes parecem pagar gorjeta igualmente



Etapa descritiva

A maioria das mesas está entre 2 e 4.

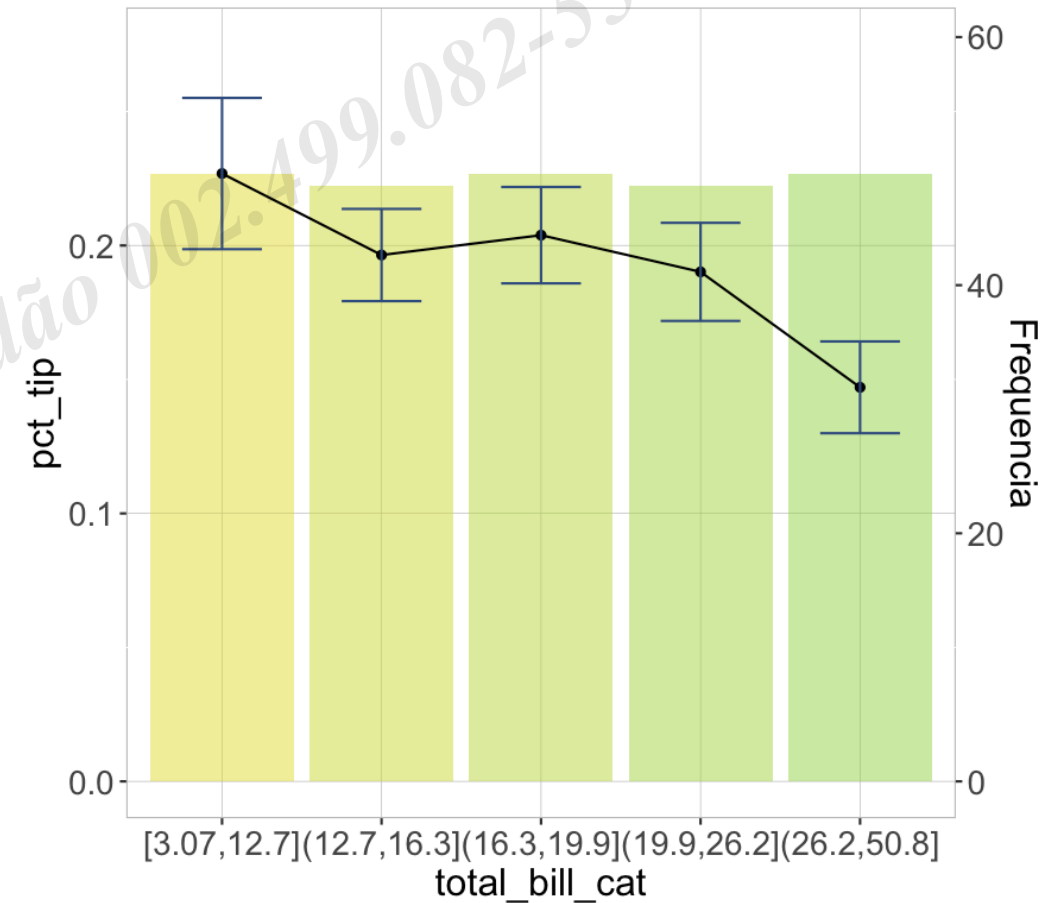
As diferenças não são muito grandes considerando a variabilidade.



Etapa descritiva

O valor da conta é uma variável contínua. Para analisar as médias, vamos dividi-la em faixas com a função quantcut.

O percentual da gorjeta parece cair quanto maior o valor da conta.



Etapa 2: treinar o modelo

2) treinar a árvore (não incluir o valor da gorjeta como explicativa)

```
arvore_reg <- rpart::rpart(pct_tip ~ sex  
  + smoker  
  + day  
  + time  
  + size,  
  data=tips,  
  xval=10,  
  control = rpart.control(cp = 0,  
    minsplit = 2,  
    maxdepth = 2)  
)
```

Etapa 2: treinar o modelo

- Ao treinar o modelo, o R define as regras da árvore conforme o algoritmo.
- Todas as informações da árvore, regras, valores de CP, hiperparâmetros e outras informações são guardados no objeto retornado
- Usaremos esse objeto retornado para realizar ações como calcular previsões, e análises como plotar a árvore.

Visualizando a árvore

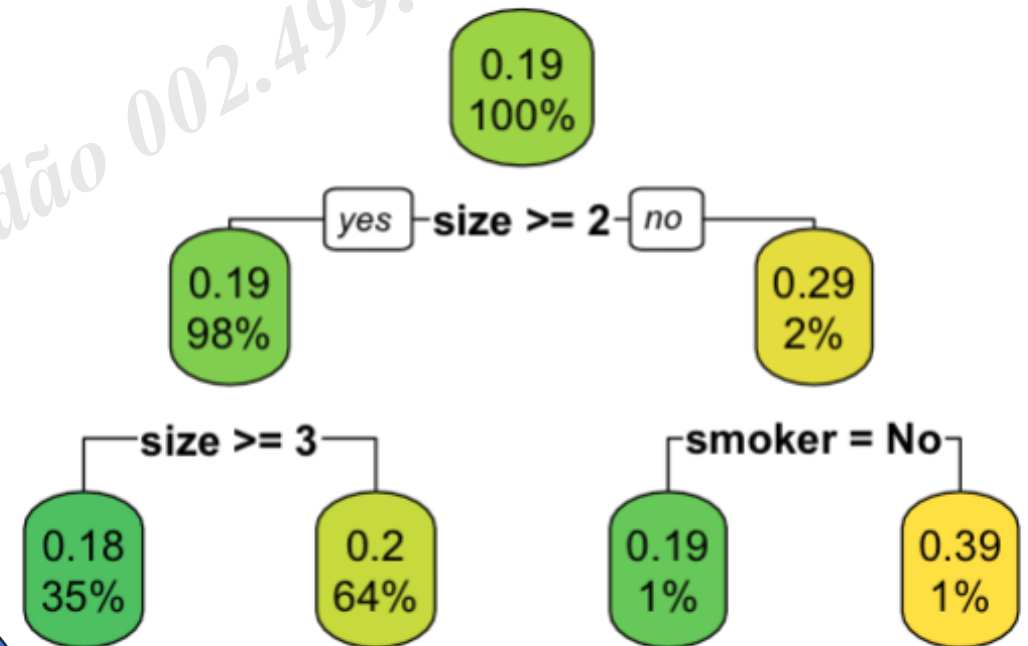
```
paleta <- scales::viridis_pal(begin=.75, end=1)(20)
```

```
#Visualizar a árvore
```

```
plot <- rpart.plot::rpart.plot(arvore_reg,  
                                box.palette = paleta)
```

R-quadrado = 6,8%

A árvore que estamos
plotando está neste
objeto



Etapa 3: Tunando a árvore

- Vamos fazer uma árvore grande com o *rpart*
- O *rpart* já implementa pra nós um *cross validation (k-fold)*
- A função *cptable* do *rpart* nos dá os valores de *cp* (complexidade do caminho) para todos os nós da árvore
- Vamos escolher o *cp* que minimiza o erro na validação cruzada e treinar a árvore novamente.

Etapa 3: treinar uma árvore grande

```
arvore_grande <- rpart::rpart(pct_tip ~ sex  
  + smoker  
  + day  
  + time  
  + size,  
  data=tips,  
  xval=10,  
  control = rpart.control(cp = 0,  
    minsplit = 2,  
    maxdepth = 30)
```

Não vamos
restringir o
tamanho da
árvore nesse
passo

Etapa 3: obtendo o CP ótimo

Calcular o R-quadrado

```
metricas(predict(arvore_grande, tips), tips$pct_tip)
```

Vamos achar o hiperparâmetro ótimo para o CP:

Esta linha guarda todos os valores de CP para cada folha da árvore grande

```
tab_cp <- rpart::printcp(arvore_grande)
```

Aqui uma visualização gráfica do CP vs erro em validação cruzada

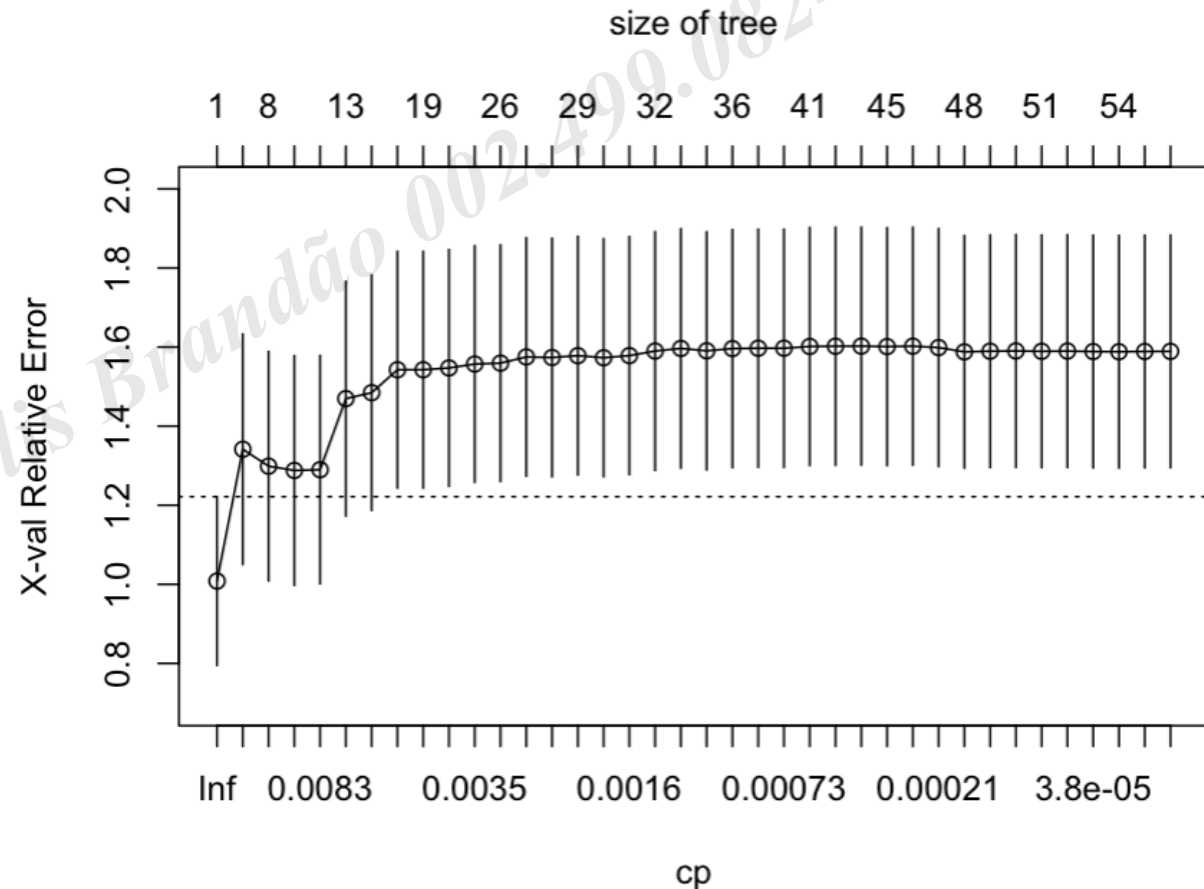
```
rpart::plotcp(arvore_grande)
```

Com esse comando vamos pegar o melhor CP na validação cruzada

```
cp_min <- tab_cp[which.min(tab_cp[, 'xerror']), 'CP']
```

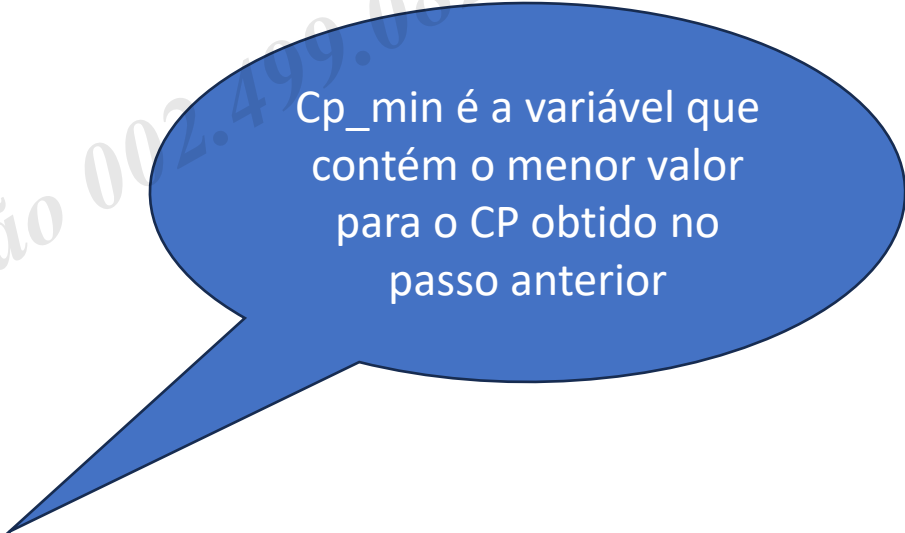
Etapa 3: obtendo o CP ótimo

Neste problema, o erro começa a aumentar logo conforme aumentamos a complexidade da árvore.



Etapa 3: Treinar a árvore tunada

```
arvore_tunada <- rpart::rpart(pct_tip ~ sex  
  + smoker  
  + day  
  + time  
  + size,  
  data=tips,  
  xval=20,  
  control = rpart.control(cp = cp_min,  
    minsplit = 2,  
    maxdepth = 30)  
)
```



Cp_min é a variável que contém o menor valor para o CP obtido no passo anterior



O problema

Queremos ‘prever’ o valor médio de residências em Boston, de acordo com características da região

A base

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax
1	0.00632	18.0	2.31	0	0.5380	6.575	65.2	4.0900	1	296
2	0.02731	0.0	7.07	0	0.4690	6.421	78.9	4.9671	2	242
3	0.02729	0.0	7.07	0	0.4690	7.185	61.1	4.9671	2	242
4	0.03237	0.0	2.18	0	0.4580	6.998	45.8	6.0622	3	222

Cada linha corresponde a uma região de Boston, cada coluna traz uma característica como índice de criminalidade, percentual da região residencial, percentual industrial, média do número de quartos, etc.

As variáveis

- # CRIM: Taxa de criminalidade per capita por região.
- # ZN: Proporção de terrenos residenciais divididos em lotes com mais de 25.000 pés quadrados (cerca de 2.322 metros quadrados).
- # INDUS: Proporção de acres não comerciais por cidade.
- # CHAS: indica se o imóvel faz fronteira com o rio Charles (1 se faz fronteira, 0 caso contrário).
- # NOX: Concentração de óxidos nítricos (partes por 10 milhões).
- # RM: Média de número de quartos por habitação.
- # AGE: Proporção de unidades ocupadas pelos proprietários construídas antes de 1940.
- # DIS: Distância ponderada até cinco centros de emprego em Boston.
- # RAD: Índice de acessibilidade a rodovias radiais.
- # TAX: Taxa de imposto sobre propriedades de valor total por \$10.000.
- # PTRATIO: Razão aluno-professor por cidade.
- # B: $1000(B_k - 0.63)^2$, onde B_k é a proporção de pessoas de origem afro-americana por cidade.
- # LSTAT: Porcentagem de status inferior da população.
- # MEDV: Valor mediano das residências ocupadas pelos proprietários em milhares de dólares. (variável de interesse)

Estratégia

- Neste exercício, vamos construir uma árvore, explorando o algoritmo e depois vamos fazer o tuning da árvore
- Em seguida vamos fazer um Random Forest, e comparar os resultados



AREM3_script02_ex2_aula_anterior_RandomForest.R

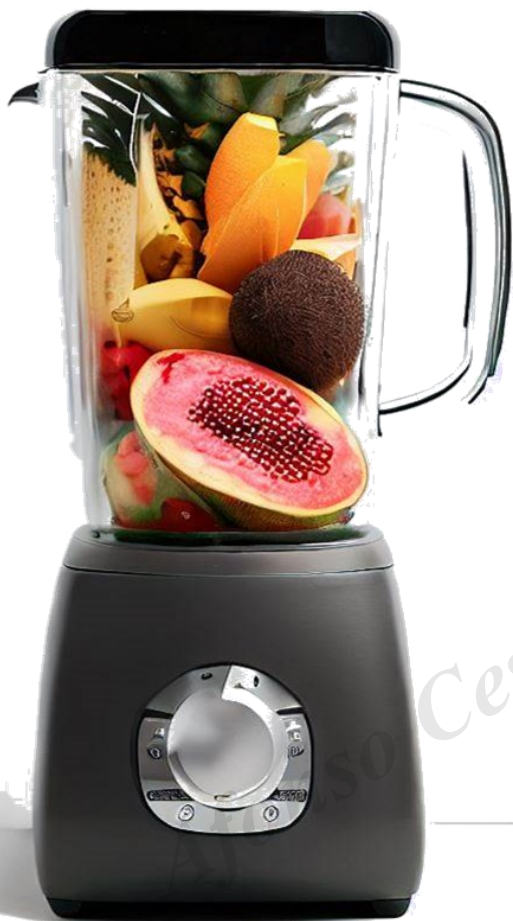


Modelos *Ensemble*:

Onde vivem? O que são?

O que comem?

Predadores naturais



Ensemble

Um ensemble é qualquer mistura de modelos já existentes.
Os principais tipos são:

Bagging

Boosting

Stacking

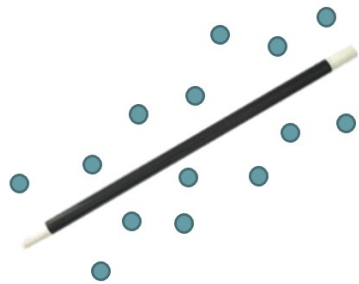
Bagging

Características (revisão)

Ensemble - aggregation

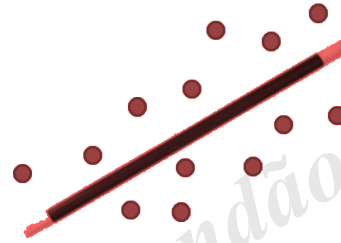


Modelo 1



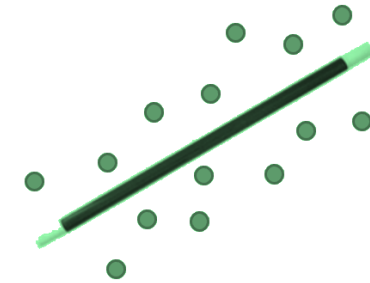
$$P(\text{blue} | \text{person}) = 3\%$$

Modelo 2



$$P(\text{blue} | \text{person}) = 7\%$$

Modelo 3

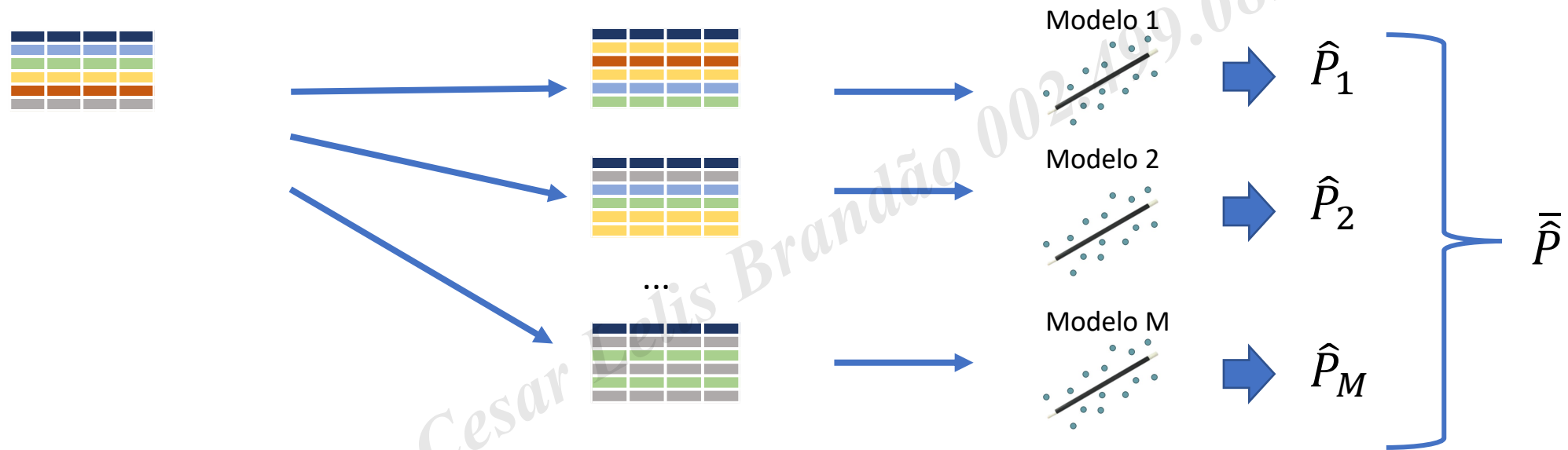


$$P(\text{blue} | \text{person}) = 2\%$$

$$P(\text{blue} | \text{person}) = (3\% + 7\% + 2\%) / 3 = 4\%$$

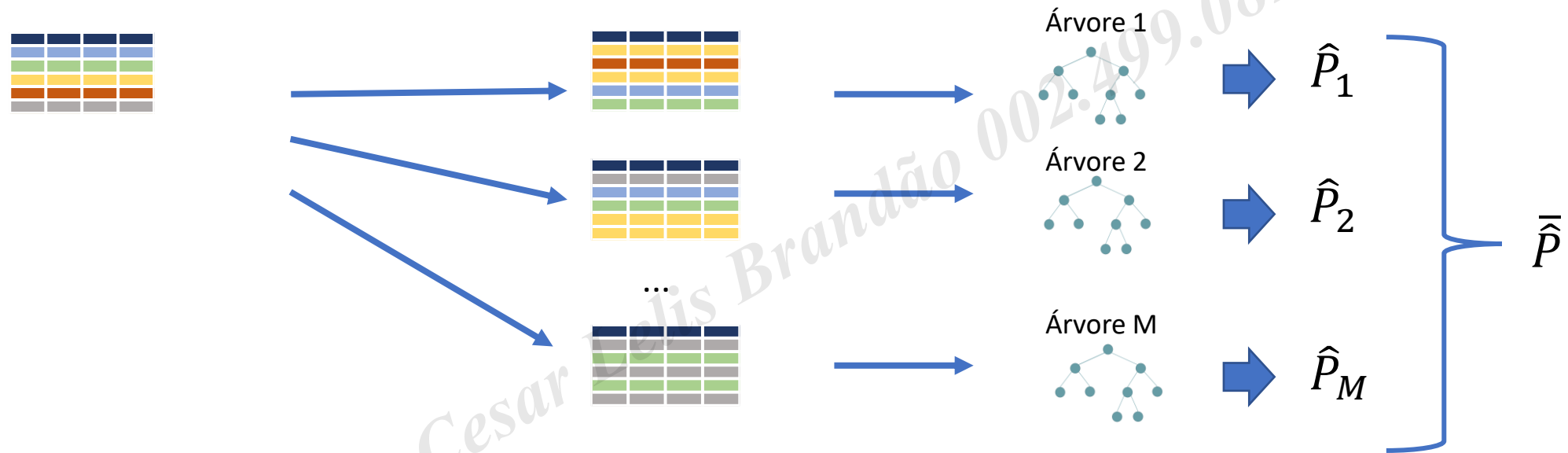
Vimos que o *aggregation* é um *ensemble* de modelos através da média das previsões.

Bootstrap – aggregation (bagging)



Vimos que o *Bagging* é um *agregation* do mesmo *algoritmo* em amostras *bootstrap*

Bootstrap – aggregation (bagging)



E que o *Random Forest* é um *bagging* com árvores

Random Forest – histórico

- Desenvolvido por Leo Breiman
- Surpreendeu ao próprio autor pela eficácia
- Possui versões posteriores com alterações e hiperparâmetros novos



Random Forest hiperparâmetros

- Mtry – o número de variáveis sorteadas a cada passo

- Controla o número de colunas sorteadas a cada passo.

id	x1	x2	x3	x4	x5
1	0,9	0,9	0,3	0,1	0
2	0,3	0,7	0,3	0,3	0,8
3	0,5	0,3	0,1	0	0,3
4	0,6	0,4	0,8	0,6	0,9
5	0,5	0,1	0,1	0,7	0,1
6	0,8	0,4	0,1	0,7	0,9



id	x1	x3	x5
1	0,6	0,8	0,5
2	0,7	0,4	0,1
3	0,4	0	0,6
4	0,6	0,9	0,1
5	1	0,4	0,3
6	0,9	0,4	0,9

- Subsample (variação) – o percentual da amostra no bootstrap

- Controla o número de linhas sorteadas a cada passo.

id	x1	x2	x3	x4	x5
1	0,7	0,1	0,8	0,1	0,1
2	0,8	0,8	0,8	0,9	0,8
3	0	0,8	0	0,7	0,3
4	0,6	0,1	1	0	0,6
5	0,2	0,3	0,6	0,5	0,6
6	0,5	0,2	0,7	0,4	0,7



id	x1	x2	x3	x4	x5
1	0,7	0,1	0,8	0,1	0,1
3	0	0,8	0	0,7	0,3
6	0,5	0,2	0,7	0,4	0,7

Random Forest hiperparâmetros

- ntree – o número de árvores utilizadas
- Parâmetros de árvore
 - Maxdepth, minsplit etc.

Stacking

Do que se trata?

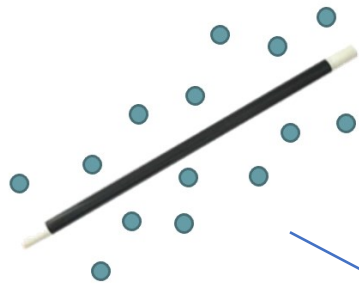


gencraft

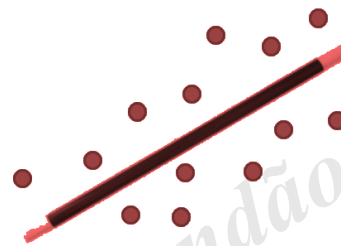
Stacking



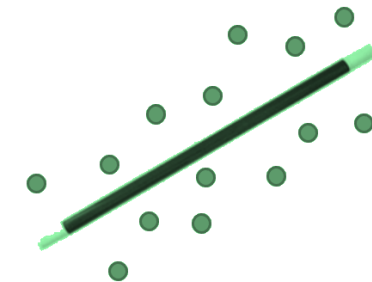
Modelo 1



Modelo 2



Modelo 3



```
1 arvore <- rpart(y ~ p1 + p2 + p3,  
2 data=df)
```

O Stacking consiste na inserção de previsões de modelos diferentes como variáveis explicativas em um novo modelo de Machine Learning. (não vamos aprofundar o stacking)

Boosting

Correção sequencial de erros

A variável resposta de uma iteração é o 'erro' da anterior.

A variável resposta de uma iteração é o 'erro' da anterior.

ID	...	Y
1	...	1
2	...	0
...	...	...
N	...	0



Y	P	ERRO
1	75%	25%
0	20%	20%
...	...	...
0	40%	40%



ERRO	Δ	P	ERRO
25%	10%	85%	15%
-20%	-10%	10%	-10%
...	...	...	...
-40%	-15%	25%	-25%



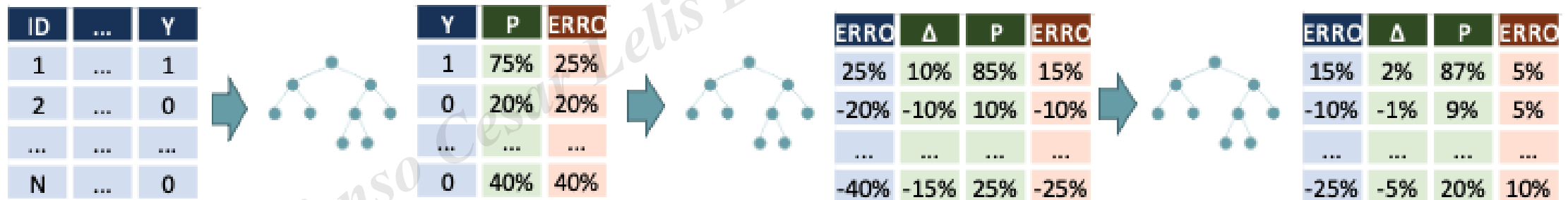
ERRO	Δ	P	ERRO
15%	2%	87%	5%
-10%	-1%	9%	5%
...	...	...	...
-25%	-5%	20%	10%

Boosting

- Os métodos de *boosting* são modelos sequenciais que tentam melhorar o erro do modelo anterior

Gradient Boosting

- O *Gradiente Boosting* um boosting baseado em árvores com alguns hiperparâmetros que controlam o algoritmo





AREM3_script03_boosting_manual.R



Tunning

"Estique a corda demais e ela arrebenta, deixe-a muito frouxa, e o instrumento não toca"

XGBoosting

Nome curto para Extreme Gradient Boosting

É uma implementação do Gradient Boosting

Possui interfaces para R e Python

Ficou famosa por ser usada por vencedores de competições

Criado por Tianqi Chen

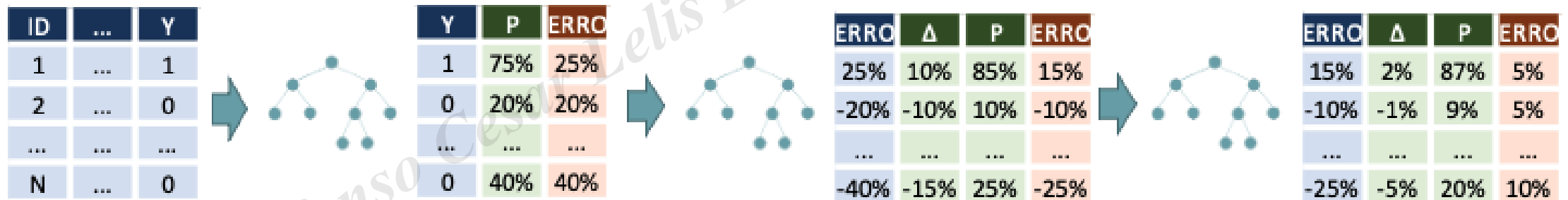
Gradient Boosting - variações

- Extreme Gradient Boosting (XGBoost)
- Uma variação do gradiente boosting
- Implementação de Tianqi Chen
- Ficou famosa por seu desempenho em competições



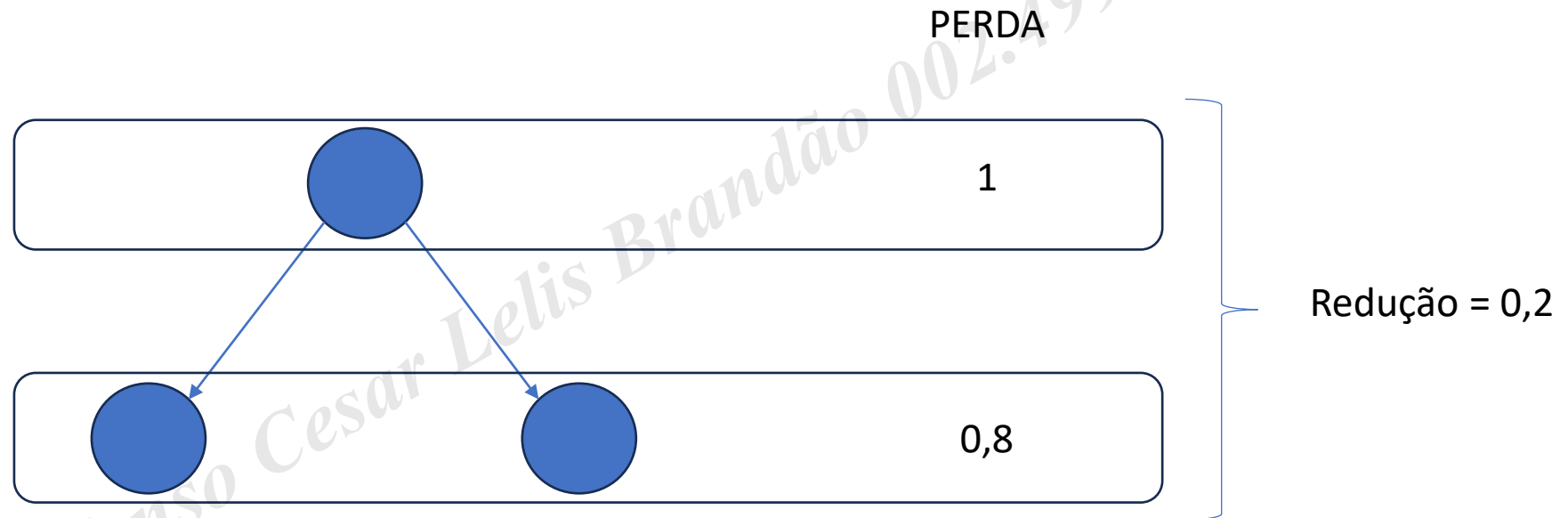
XGBoost - hiperparâmetros

- Nrounds – o número de árvores sequenciais



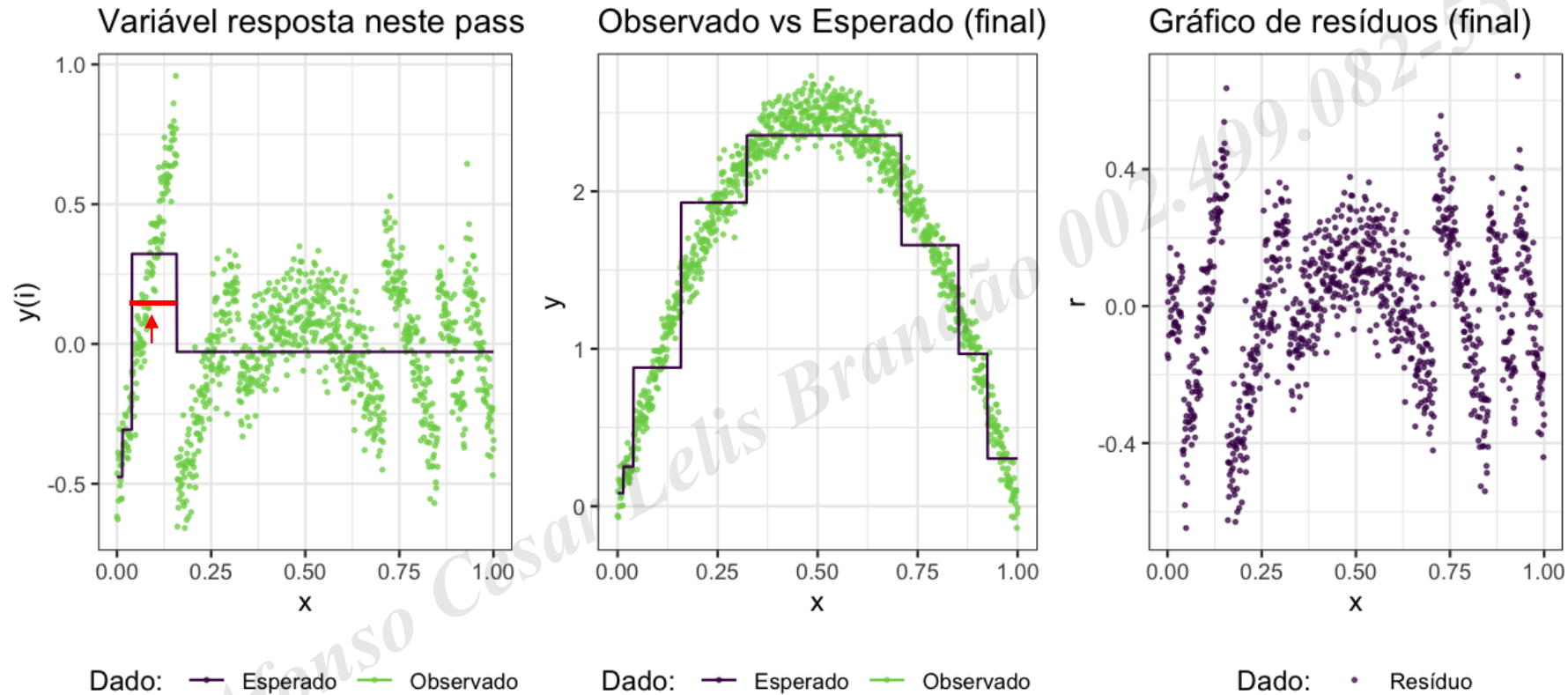
XGBoost - hiperparâmetros

- Gamma – redução mínima de perda



Se usa em geral valores de gamma entre 0 e 10.
O valor ótimo pode variar muito de um problema para outro.

XGBoost - eta



O Learning Rate diminui o impacto de cada iteração
costuma demandar mais iterações,
mas ajuda a alcançar melhores resultados

XGBoost - hiperparâmetros

- Colsample_bytree – o número de variáveis sorteadas a cada passo

- Controla o número de colunas sorteadas a cada passo.

id	x1	x2	x3	x4	x5
1	0,9	0,9	0,3	0,1	0
2	0,3	0,7	0,3	0,3	0,8
3	0,5	0,3	0,1	0	0,3
4	0,6	0,4	0,8	0,6	0,9
5	0,5	0,1	0,1	0,7	0,1
6	0,8	0,4	0,1	0,7	0,9



id	x1	x3	x5
1	0,6	0,8	0,5
2	0,7	0,4	0,1
3	0,4	0	0,6
4	0,6	0,9	0,1
5	1	0,4	0,3
6	0,9	0,4	0,9

- Subsample (variação) – o percentual da amostra no bootstrap

- Controla o número de linhas sorteadas a cada passo.

id	x1	x2	x3	x4	x5
1	0,7	0,1	0,8	0,1	0,1
2	0,8	0,8	0,8	0,9	0,8
3	0	0,8	0	0,7	0,3
4	0,6	0,1	1	0	0,6
5	0,2	0,3	0,6	0,5	0,6
6	0,5	0,2	0,7	0,4	0,7



id	x1	x2	x3	x4	x5
1	0,7	0,1	0,8	0,1	0,1
3	0	0,8	0	0,7	0,3
6	0,5	0,2	0,7	0,4	0,7

XGBoosting - hiperparâmetros

- Nrounds – número de árvores sequenciais
- Max_depth – profundidade máxima de cada árvore
- Gamma – redução mínima de perda necessária para realizar uma quebra
- Eta – Taxa de aprendizado. Uma penalização no ‘totozinho’ a cada passo.
- Colsample_bytree – número de colunas sorteadas por ‘round’
- Min_child_weight – número mínimo de observações por folha
- Subsample – percentual de amostragem para o *bootstrap*



AREM3_script04_XGBoost.R

HAR



- Identificação de atividade humana por acelerômetro e giroscópio do celular
- Dados originais captados com frequência de 40 Hz
- Dados sumarizados em 1 medida a cada 1,4 segundos
- Medidas de translação e rotação nos eixos X, Y e Z
- Derivações como aceleração, *jerk*, médias, máximos, mínimos, desvios etc dentro das janelas.

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

HAR

1 - andando



2 – subindo escada



3 – descendo escada



4 - sentado



5 – em pé



6 – deitado



HAR - Variáveis

Medidas

tBodyAcc-XYZ	tBodyGyroMag
tGravityAcc-XYZ	tBodyGyroJerkMag
tBodyAccJerk-XYZ	fBodyAcc-XYZ
tBodyGyro-XYZ	fBodyAccJerk-XYZ
tBodyGyroJerk-XYZ	fBodyGyro-XYZ
tBodyAccMag	fBodyAccMag
tGravityAccMag	fBodyAccJerkMag
tBodyAccJerkMag	fBodyGyroMag
	fBodyGyroJerkMag

Resumos

Média
Desvio padrão
Mediana
Mínimo
Máximo
SMA
Energia
IQR
Entropia
etc

Total

561 atributos

HAR - base

	V1	X1tBodyAcc.mean.X	X2tBodyAcc.mean.Y	X3tBodyAcc.mean.Z	X4tBodyAcc.mean.X
1	em_pe	0.28858451	-0.0202941710	-0.13290514	-0.00000000
2	em_pe	0.27841883	-0.0164105680	-0.12352019	-0.00000000
3	em_pe	0.27965306	-0.0194671560	-0.11346169	-0.00000000
4	em_pe	0.27917394	-0.0262006460	-0.12328257	-0.00000000
5	em_pe	0.27662877	-0.0165696550	-0.11536185	-0.00000000
6	em_pe	0.27719877	-0.0100978500	-0.10513725	-0.00000000
7	em_pe	0.27945388	-0.0196407760	-0.11002215	-0.00000000
8	em_pe	0.27743247	-0.0304883030	-0.12536043	-0.00000000
9	em pe	0.27729342	-0.0217506980	-0.12075082	-0.00000000

- Cada linha é uma medida
- Há várias medidas por pessoa



AREM3_script04_XGBoost.R

O que fazer com meus novos superpoderes?

- Exercícios para a próxima:
 - Melhorar o *fit* na base HAR

Afonso Cesar Lelis Brandão



Conclusões

- Árvores são só o começo... Ou UM começo
- Há INFINITAS formas de combinar modelos, essas são as mais famosas
- Esses modelos são difíceis de se interpretar
- XGB é um modelo delicado que toma tempo de tuning
- *PRATIQUE!*





elis Brandão 002.499.082-55

Por hoje é só ;)



[linkedin.com/in/joao-serrajordia](https://www.linkedin.com/in/joao-serrajordia)

OBRIGADO!

[linkedin.com/in/joao-serrajordia/](https://www.linkedin.com/in/joao-serrajordia/)